

Formalizing Privacy-Enhanced Whitelists: A Secure Framework with Hidden Policies

Yu Zhang^{1,2}
Zongbin Wang³

¹Northwestern Polytechnical University

²Beijing Pushtime Technology Co., Ltd.

³Beijing Infosec Technologies Co., Ltd.

Email: zy168612@163.com; pony_wang@hotmail.com

Abstract

The whitelist is a foundational and widely deployed access control mechanism. In its prevalent implementation, the verifying entity typically requires access to the plaintext authorization policy to perform enforcement. This creates a concentrated security risk: the verifier becomes a high-value target, and its compromise could lead to the full exposure of the sensitive whitelist—a single point of failure for policy confidentiality. This work formalizes and addresses this risk by introducing a new paradigm of Privacy-Enhanced Whitelists. Our framework cryptographically decouples policy enforcement from policy exposure. A trusted issuer encodes the whitelist, enabling the verifier to validate memberships using only cryptographically blinded commitments, without ever needing the plaintext list. We provide a formal security model that rigorously defines and achieves security even against an adversary who compromises the verifier’s state. The system is realized via an efficient construction based on polynomial commitments in elliptic curve groups and supports dynamic policy updates. By shifting the trust assumption from the verifier’s infrastructure to cryptographic verification, our work establishes a practical foundation for scenarios where the whitelist itself must remain a protected secret.

Keywords

whitelist, access control, privacy, formal model, pattern-based

ACM Reference Format:

Yu Zhang^{1,2} and Zongbin Wang³, ¹Northwestern Polytechnical University, ²Beijing Pushtime Technology Co., Ltd., ³Beijing Infosec Technologies Co., Ltd., Email: zy168612@163.com; pony_wang@hotmail.com . . Formalizing Privacy-Enhanced Whitelists: A Secure Framework with Hidden Policies. In *Proceedings of* . ACM, New York, NY, USA, 11 pages.

1 Introduction

Access control systems[1–3] form the foundational perimeter of information security, determining which entities may interact with protected resources under specified conditions. Among the spectrum of access control models, the *whitelist* (or allowlist) paradigm is distinguished by its implementation of a strict default-deny principle, wherein only a pre-defined set of trusted entities—either explicitly enumerated or described by a pattern-based rule—is granted authorization. This model provides deterministic and auditable security guarantees, making it indispensable for securing critical infrastructures such as network perimeters, secure boot sequences,

trusted execution environments, and high-assurance software supply chains[4–6].

However, the conventional operational paradigm of whitelist-based access control introduces a profound security paradox. The fundamental mechanism of enforcement inherently requires the *verifier*—the entity executing the access decision—to possess a complete, plaintext representation of the authorization policy for verification. This architectural necessity thus elevates the verifier to a high-value target, as its compromise would directly yield the full authorization policy—a realistic threat given its exposed position within the security architecture. The disclosure of whitelist may lead to severe systemic consequences. Existing approaches for enhancing whitelist privacy—utilizing encryption or hashing—while useful, do not fully resolve this core tension between effective access control and policy confidentiality.

Therefore, how to achieve both expressive policy enforcement and provable policy confidentiality constitutes the primary motivation of this work.

1.1 Related Work

The problem of constructing a privacy-enhanced whitelist(PEW) bears a surface resemblance to, but is fundamentally distinct from, the classical Private Set Intersection (PSI) problem. PSI protocols[7–14] enable two mutually distrustful parties, each holding a private input set, to learn only the intersection of their sets. This model is designed for scenarios of mutual privacy, such as private contact discovery, where both parties’ inputs are sensitive and the output (the intersection) is shared.

While both paradigms involve computing an output based on a hidden set, their core security objectives and adversarial models diverge fundamentally. In PEW, the primary adversary is external and may compromise the verifier’s state. The security goal thus becomes protecting the policy’s confidentiality from the verifier itself, even after compromise. This establishes a strict, asymmetric trust model and a requirement for persistent secrecy of a dynamically updatable set—requirements orthogonal to the mutual privacy and peer-to-peer computation of standard PSI.

A direct method to enhance whitelist privacy is to store it in encrypted form at the verifier[15]. For verification, the verifier decrypts the list to compare against the candidate identifier in plaintext. This protects the policy *in storage*, but not *during computation*: the decryption key remains a single point of failure, and plaintext identifiers are exposed in memory during verification.

Consequently, encryption alone does not achieve policy confidentiality against a compromised verifier.

An alternative is to store only cryptographically hashed identifiers (e.g., $H(id_i)$) at the verifier[16]. To verify a candidate id^* , the verifier computes $H(id^*)$ and checks for a match in the stored hash list. This method protects the plaintext whitelist both *in storage* and *during computation*, as the hash values alone are exposed. However, its expressiveness is fundamentally limited to exact equality checks. It cannot natively support more expressive, *pattern-based* matching policies (e.g., ranges, prefixes). Thus, while providing a baseline level of privacy, hash-based schemes lack the flexibility required for complex, real-world authorization scenarios.

This work aims to address the above limitations by constructing a cryptographic framework that achieves provable policy confidentiality against a possible compromised verifier, while supporting expressive authorization beyond simple equality checks.

1.2 Contributions

This paper makes the following key contributions:

(1) A Formal Model for Privacy-Enhanced Access Control.

We introduce the first formal model for Privacy-Enhanced Access Control systems, with a focus on whitelist enforcement. The model defines a unified syntax for policy issuance, verification, and trusted updates, capturing both simple membership lists and more expressive, **pattern-based policies**. Its core security property, *Policy Confidentiality against a Compromised Verifier*, guarantees that an adversary who compromises the verifier learns no more about the policy—whether a list or a pattern—than what is revealed by observed verification queries.

(2) A Concrete and Efficient Cryptographic Construction.

We instantiate the model with an efficient, unified scheme based on polynomial commitments. A trusted issuer encodes the authorization policy—be it a specific set of identifiers or a pattern admitting a compact algebraic representation—and provides the verifier with a vector of cryptographic commitments. The scheme elegantly separates steady-state operation from update procedures: for routine checks, the verifier performs **non-interactive policy evaluations** via secure exponentiation without accessing plaintext; for updates, the issuer triggers a refresh of this commitment. This design ensures the verifier’s persistent state contains no plaintext policy. Crucially, **transient exposure of plaintext identifiers is not an inherent consequence of the update mechanism**; it is confined to specific implementation choices, and even then can be mitigated through system-level protections. This design allows for a range of privacy-efficiency trade-offs.

(3) Security Analysis.

We provide rigorous security proofs demonstrating that our construction satisfies the defined policy confidentiality under standard cryptographic assumptions (specifically, the Discrete Logarithm assumption). The proofs formally reduce the security of our scheme to the hardness of the computational problem in the underlying group, covering both exact-match and pattern-based policy instantiations within the unified framework.

(4) Implementation and Evaluation.

We conduct a comprehensive performance evaluation through a proof-of-concept implementation. Our experiments measure the computational overhead for issuance, verification, and policy updates across representative policy types, including exact whitelists and common patterns. The results demonstrate the practical feasibility and low latency of our unified scheme for deployment at realistic scales, validating its expressiveness and efficiency.

2 Preliminaries

This section reviews the cryptographic background necessary for understanding our construction.

2.1 Cyclic Groups and Hardness Assumption

Let $\mathbb{G}$ be a multiplicative cyclic group of prime order p , where the bit-length of p is determined by the security parameter λ . We assume $\mathbb{G}$ is instantiated with a suitable elliptic curve. Let g be a fixed generator of $\mathbb{G}$. The *Discrete Logarithm (DL)*[17–21] assumption states that for any probabilistic polynomial-time (PPT) adversary $\mathcal{A}$, the following advantage is a negligible function in λ :

$$\text{Adv}_{\mathbb{G}}^{\text{DL}}(\mathcal{A}) = \Pr \left[a \xleftarrow{\$} \mathbb{Z}_p; h = g^a : \mathcal{A}(\mathbb{G}, g, h) = a \right].$$

The security of our construction relies on the hardness of the DL problem in $\mathbb{G}$.

2.2 Polynomials over Finite Fields

Let p be a prime. We denote by $\mathbb{Z}_p[X]$ the ring of polynomials in the indeterminate X with coefficients in the finite field $\mathbb{Z}_p$. Let $f(X) = \sum_{k=0}^n a_k X^k$ be such a polynomial, where $a_k \in \mathbb{Z}_p$. For any element $x \in \mathbb{Z}_p$, we denote by $f(x) \in \mathbb{Z}_p$ the evaluation of the polynomial f at x , computed as $\sum_{k=0}^n a_k x^k$.

For a set $S = \{id_1, \dots, id_n\} \subset \mathbb{Z}_p$ of n distinct elements, we define the unique monic polynomial of degree n that has S as its set of roots:

$$f_S(X) = \prod_{id \in S} (X - id) \in \mathbb{Z}_p[X].$$

A key property used throughout our work is that for any $id^* \in \mathbb{Z}_p$, $f_S(id^*) = 0$ if and only if $id^* \in S$.

2.3 Cryptographic Polynomial Commitments in the Exponent

Our scheme uses a polynomial commitment scheme[22–24] where coefficients are committed *in the exponent*. Given a polynomial $f(X) = \sum_{k=0}^n a_k X^k \in \mathbb{Z}_p[X]$, the commitment is the vector of group elements:

$$\text{Com}(f) = (g^{a_0}, g^{a_1}, \dots, g^{a_n}) \in \mathbb{G}^{n+1}.$$

For any input $x \in \mathbb{Z}_p$, the evaluation $f(x)$ can be computed homomorphically *in the exponent* using the commitment:

$$g^{f(x)} = g^{\sum_{k=0}^n a_k x^k} = \prod_{k=0}^n (g^{a_k})^{x^k}.$$

2.4 Range Representation via Best Range-Covering

Our scheme employs the *Best Range-Covering (BRC)* technique [25, 26]. This method compactly represents a continuous numerical interval using a minimal set of nodes from a perfect binary tree.

Binary Tree Model. Consider a perfect binary tree of height m with 2^m leaves. The leaf nodes, from left to right, correspond to the numerical domain $D = \{0, 1, \dots, 2^m - 1\}$. Each leaf τ_i is assigned the value i . A non-leaf node is denoted by $\tau_{i,j}$, where i and j are the values of its leftmost and rightmost descendant leaves, respectively. This node $\tau_{i,j}$ conceptually *covers* the contiguous range of values $\{i, i + 1, \dots, j\} \subseteq D$.

Best Range-Covering (BRC). Given a range $[a, b] \subseteq D$, its *Best Range-Covering* is defined as the **minimum set of tree nodes** whose combined covered ranges exactly equal $[a, b]$. This representation is both compact and canonical.

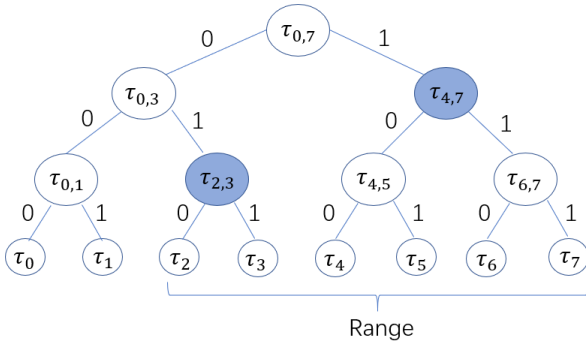


Figure 1: Example of BRC for range $[2, 7]$ in a tree of height $m = 3$ ($D = \{0, \dots, 7\}$).

Example. As illustrated in Fig. 1, for $m = 3$ and range $[2, 7]$, the BRC is $\{\tau_{2,3}, \tau_{4,7}\}$. For algorithmic convenience, nodes in the BRC are often represented by their *root-to-node path* as a binary string (e.g., 01 for $\tau_{2,3}$ and 1 for $\tau_{4,7}$ in the example, yielding the set $\{01, 1\}$).

Algorithm. We rely on the standard GetBRC algorithm [27, 28] to compute this covering.

$$\text{GetBRC}(a, b, m) \rightarrow \mathcal{B}$$

The algorithm takes as input the tree height m and a range $[a, b] \subseteq [0, 2^m - 1]$, and outputs the BRC $\mathcal{B}$ (a set of node identifiers) for $[a, b]$. This compact representation is foundational for efficiently encoding range policies in our cryptographic construction.

Range Membership via BRC. To determine whether a specific value $c \in D$ lies within a range $[a, b]$, we leverage a property of the BRC representation. First, we define an auxiliary function $\text{GetBinPrefixes}(c, m) \rightarrow \mathcal{P}_c$, which returns the set of all binary prefixes of c 's m -bit representation, including the full string. For example, for $m = 3$ and $c = 4$ (binary 100), we have $\text{GetBinPrefixes}(4, 3) = \{1, 10, 100\}$.

The key membership test is then characterized by the following condition: a value c belongs to the range $[a, b]$ if and only if the set

of its binary prefixes $\mathcal{P}_c$ intersects with the BRC $\mathcal{B}$ of that range. Formally,

$$c \in [a, b] \iff \mathcal{P}_c \cap \mathcal{B} \neq \emptyset.$$

This property provides an efficient way to check range membership by comparing two compact sets (prefixes and BRC nodes), which forms the basis for integrating range policies into our cryptographic construction.

3 Problem Statement and System Model

3.1 Problem Statement

We address the foundational challenge of designing an access control system that enforces authorization policies while preserving their confidentiality. A trusted **Issuer** defines the policy, formally represented as a predicate $\Phi : \mathcal{ID} \rightarrow \{0, 1\}$ over the identifier space $\mathcal{ID}$. A policy can be an **explicit** enumeration (e.g., a whitelist S) or a **pattern-based** rule R (e.g., specifying a prefix or a range). A separate **Verifier** must perform non-interactive access decisions against the hidden predicate Φ , while its persistent state may be compromised by an external adversary. The system must also support policy updates initiated by the Issuer.

Existing privacy-enhancing approaches (e.g., encryption or hashing) do not fully resolve the tension between policy expressiveness and confidentiality. Therefore, achieving both *expressive policy enforcement* and *provable policy confidentiality* forms the core motivation of this work. We formalize and address the security requirement of *policy confidentiality against a compromised Verifier*: even if an adversary obtains the Verifier's complete internal state, it should not learn the policy predicate Φ .

3.2 System Model

Our system model, illustrated in Figure 2, formally defines the entities, their roles, and the algorithms comprising a Privacy-Enhanced Whitelist (PEW) system.

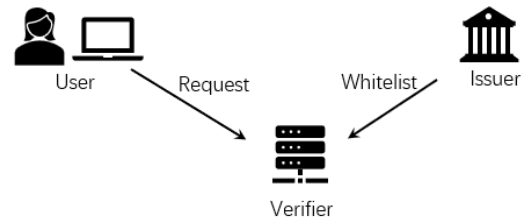


Figure 2: System architecture and interaction flow.

Entities. The system comprises three roles:

- **Issuer ($\mathcal{I}$):** A fully trusted entity that initializes the system, defines the authorization policy (whitelist S or rule R), and authorizes all subsequent updates.
- **Verifier ($\mathcal{V}$):** The entity that enforces access control. It receives and stores a cryptographic representation of the policy from $\mathcal{I}$ and executes the verification protocol to grant or deny access. $\mathcal{V}$ is assumed to follow the protocol honestly, but its persistent internal state may be compromised by an external adversary.

- **User ($\mathcal{U}$):** An entity that requests access by submitting a candidate identifier id^* to $\mathcal{V}$ for verification.

Algorithms. A PEW scheme consists of the following polynomial-time algorithms:

- **Setup(1^λ) $\rightarrow pp$:** On input the security parameter λ , $\mathcal{I}$ generates public parameters pp .
- **Issue(pp, Φ) $\rightarrow st_{\mathcal{V}}$:** Executed by $\mathcal{I}$. On input the public parameters pp and a policy predicate Φ , it outputs the initial verifier state $st_{\mathcal{V}}$, which encodes Φ in a hidden form. $\mathcal{I}$ transmits $st_{\mathcal{V}}$ to $\mathcal{V}$ to bootstrap the system.
- **Verify($pp, st_{\mathcal{V}}, id^*$) $\rightarrow \{0, 1\}$:** Executed by $\mathcal{V}$. On input pp , state $st_{\mathcal{V}}$, and a candidate identifier $id^* \in \mathcal{ID}$, it returns 1 if id^* satisfies the policy Φ encoded in $st_{\mathcal{V}}$, and 0 otherwise.
- **Update($pp, st_{\mathcal{V}}, \Delta$) $\rightarrow st'_{\mathcal{V}}$:** Executed by $\mathcal{V}$ to transition the system to a new policy state. It takes pp , the current state $st_{\mathcal{V}}$, and an *update instruction* Δ authorized by $\mathcal{I}$. The instruction Δ may contain explicit cryptographic material or the necessary plaintext data to locally compute the new state. The algorithm outputs the updated state $st'_{\mathcal{V}}$ corresponding to the modified policy.

The communication channel between the Issuer and the Verifier for the initial distribution of $st_{\mathcal{V}}$ must provide authenticity and integrity, but need not be confidential, as $st_{\mathcal{V}}$ constitutes a public commitment. The security of specific update mechanisms may rely on additional, refined channel assumptions, which will be discussed alongside their respective constructions.

3.3 Formal Security Definitions

We now formally define the security properties for a Privacy-Enhanced Whitelist (PEW) scheme. Let $\Pi = (\text{Setup}, \text{Issue}, \text{Verify}, \text{Update})$ denote a PEW scheme as defined in Section 3.2. All adversaries are modeled as probabilistic polynomial-time (PPT) algorithms.

3.3.1 Completeness. Completeness guarantees that any identifier satisfying the policy will be accepted. For any policy Φ of polynomial-size representation and any identifier $id \in \mathcal{ID}$ such that $\Phi(id) = 1$, the following holds with overwhelming probability over the randomness of the algorithms:

$$\Pr \left[\begin{array}{l} pp \leftarrow \text{Setup}(1^\lambda), \\ st_{\mathcal{V}} \leftarrow \text{Issue}(pp, \Phi), \\ b \leftarrow \text{Verify}(pp, st_{\mathcal{V}}, id) : b = 1 \end{array} \right] \geq 1 - \text{negl}(\lambda).$$

3.3.2 Soundness. Soundness guarantees that any identifier not satisfying the policy will be rejected. For any policy Φ of polynomial-size representation and any identifier $id \in \mathcal{ID}$ such that $\Phi(id) = 0$, the probability of a false acceptance is negligible:

$$\Pr \left[\begin{array}{l} pp \leftarrow \text{Setup}(1^\lambda), \\ st_{\mathcal{V}} \leftarrow \text{Issue}(pp, \Phi), \\ b \leftarrow \text{Verify}(pp, st_{\mathcal{V}}, id) : b = 1 \end{array} \right] \leq \text{negl}(\lambda).$$

3.3.3 Correctness of Dynamic Updates.

Definition 3.1 (Update Correctness). A PEW scheme satisfies *update correctness* if for all security parameters λ , all policies Φ of polynomial-size representation, and all valid update instructions Δ

authorized by the issuer, the following holds with overwhelming probability over the algorithms' randomness:

Let $pp \leftarrow \text{Setup}(1^\lambda)$ and $st_{\mathcal{V}} \leftarrow \text{Issue}(pp, \Phi)$. Let $st'_{\mathcal{V}} \leftarrow \text{Update}(pp, st_{\mathcal{V}}, \Delta)$ denote the updated state, and let Φ' be the policy after applying Δ to Φ . Then for all $id \in \mathcal{ID}$, it must hold that

$$\text{Verify}(pp, st'_{\mathcal{V}}, id) = \text{Verify}(pp, st''_{\mathcal{V}}, id),$$

where $st''_{\mathcal{V}} \leftarrow \text{Issue}(pp, \Phi')$ is a fresh issuance of the updated policy.

3.3.4 Policy Confidentiality against Compromised Verifier. This is the core privacy property. It ensures that even if an adversary obtains the verifier's persistent state $st_{\mathcal{V}}$, it learns no additional information about the policy Φ beyond what can be inferred from a bounded number of legitimate verification queries it is allowed to make. We define this via the following game $\text{Exp}_{\mathcal{A}, \Pi}^{\text{PC}}(\lambda)$ between a challenger C and an adversary $\mathcal{A}$:

- (1) **Setup:** The challenger C runs $pp \leftarrow \text{Setup}(1^\lambda)$ and provides pp to $\mathcal{A}$.
- (2) **Challenge:**
 - (a) $\mathcal{A}$ specifies:
 - (i) Policy type $\Phi_{\text{type}} \in \{\text{explicit}, \text{pattern-based}\}$
 - (ii) Size parameter $n = \text{poly}(\lambda)$ with $1 \leq n \leq 2^m$, where m is the bit-length of identifiers (so $|\mathcal{ID}| = 2^m$).
 - (b) C uniformly selects a policy Φ of type Φ_{type} that accepts exactly n identifiers.
 - (c) C runs $st_{\mathcal{V}} \leftarrow \text{Issue}(pp, \Phi)$ and gives the verifier state $st_{\mathcal{V}}$ to $\mathcal{A}$.
- (3) **Local Query Phase:** Upon receiving $st_{\mathcal{V}}$, adversary $\mathcal{A}$ can locally and adaptively execute $\text{Verify}(pp, st_{\mathcal{V}}, \cdot)$ for any polynomial number of identifiers id_j of its choice.
- (4) **Final Output:** After its queries, $\mathcal{A}$ outputs id^* . The game outputs 1 if $\Phi(id^*) = 1$ (i.e., id^* satisfies the policy), and 0 otherwise.

The advantage of adversary $\mathcal{A}$ is defined as:

$$\text{Adv}_{\mathcal{A}, \Pi}^{\text{PC}}(\lambda) = \Pr [\text{Exp}_{\mathcal{A}, \Pi}^{\text{PC}}(\lambda) = 1]$$

Definition 3.2 (Policy Confidentiality). A PEW scheme Π satisfies *Policy Confidentiality against a Compromised Verifier* if for all PPT adversaries $\mathcal{A}$, the advantage $\text{Adv}_{\mathcal{A}, \Pi}^{\text{PC}}(\lambda)$ is negligible in λ .

4 Instantiating the Formal Model

This section presents a concrete cryptographic construction that instantiates the formal Privacy-Enhanced Whitelist (PEW) model defined in Section 3.2 and satisfies the security notions from Section 3.3.

4.1 System Setup

The Setup algorithm initializes the system's cryptographic foundation. It is run once by the Issuer.

Algorithm 1 Setup(1^λ)**Input:** Security parameter λ **Output:** Public parameters pp

- 1: Choose a cyclic group $\mathbb{G}$ of prime order p such that the Discrete Logarithm problem in $\mathbb{G}$ provides λ -bits of security.
- 2: Let g be a fixed generator of $\mathbb{G}$.
- 3: **return** $pp \leftarrow (\mathbb{G}, p, g)$

The parameters pp are published and used by all subsequent algorithms.

4.2 Policy Commitment

The Issue algorithm is executed by the Issuer to encode an authorization policy into a cryptographic commitment. The policy Φ can be either an explicit whitelist $S = \{id_1, \dots, id_n\} \subset \mathbb{Z}_p$ or a pattern-based rule R . For pattern-based rules, we first convert them into a canonical representation using the Best Range-Covering (BRC) technique as described in Section 2.4.

Algorithm 2 Issue(pp, Φ)**Input:** Public parameters $pp = (\mathbb{G}, p, g)$, Policy Φ **Output:** Initial verifier state st_V

- 1: **if** Φ is an explicit whitelist $S = \{id_1, \dots, id_n\} \subset \mathbb{Z}_p$ **then**
- 2: Compute $f_\Phi(X) = \prod_{id \in S} (X - id) = \sum_{k=0}^n a_k X^k$, where $a_n = 1$.
- 3: **else** ▷ Pattern-based policy
- 4: **if** R is a numeric range $[a, b]$ **then**
- 5: $\mathcal{B} \leftarrow \text{GetBRC}(a, b, m)$ ▷ m is the bit-length of identifiers
- 6: **else if** R is a prefix rule P (e.g., “192.168.9.*”) **then**
- 7: Let ℓ be the length of the fixed prefix P (e.g., $\ell = 24$ for “192.168.9.*”)
- 8: Convert P to interval $[a, b]$: $a = P \parallel 0^{m-\ell}$, $b = P \parallel 1^{m-\ell}$ (e.g., $a = 0xC0A80900$, $b = 0xC0A809FF$ for $m = 32$)
- 9: $\mathcal{B} \leftarrow \text{GetBRC}(a, b, m)$
- 10: **end if**
- 11: Compute $f_\Phi(X) = \prod_{brc \in \mathcal{B}} (X - brc) = \sum_{k=0}^n a_k X^k$, where $n = |\mathcal{B}|$ and $a_n = 1$.
- 12: **end if**
- 13: **Optional:** Sample $r \xleftarrow{\$} \mathbb{Z}_p^*$ and compute $a'_k \leftarrow a_k \cdot r \bmod p$ for $k = 0, \dots, \deg(f_\Phi)$.
- 14: **for** $k = 0$ to $\deg(f_\Phi)$ **do**
- 15: $g_k \leftarrow g^{a'_k}$ if randomization used, else $g_k \leftarrow g^{a_k}$
- 16: **end for**
- 17: Assemble the verifier state $st_V \leftarrow (g_0, g_1, \dots, g_{\deg(f_\Phi)})$.
- 18: **return** st_V

The Issuer transmits the commitment vector st_V to the Verifier. Note that suffix patterns (e.g., “*@gmail.com”) can be supported by reversing the identifier string (e.g., to “moc.liamg@”) and treating it as a prefix pattern.

4.3 Membership Verification

The Verify algorithm, defined in Algorithm 4, is executed by the Verifier to test whether a candidate identifier satisfies the policy. The

algorithm first checks the policy type(Φ_{type}): for explicit enumeration policies (whitelist S), it directly calls VerifyPoly (Algorithm 3) to evaluate the polynomial at the candidate identifier. For pattern-based policies (range/prefix/suffix), it enumerates all binary prefixes of the candidate identifier using GetBinPrefixes and evaluates the polynomial at each prefix via VerifyPoly, accepting if any evaluation returns 1.

Algorithm 3 VerifyPoly(pp, st_V, x)**Input:** Public parameters $pp = (\mathbb{G}, p, g)$, Verifier state $st_V = (g_0, g_1, \dots, g_n)$, evaluation point $x \in \mathbb{Z}_p$ **Output:** Binary decision $b \in \{0, 1\}$ (1 indicates $f_\Phi(x) = 0$)

- 1: Initialize $E \leftarrow 1_{\mathbb{G}}$, $pow \leftarrow 1$
- 2: **for** $k = 0$ to n **do**
- 3: $E \leftarrow E \cdot g_k^{pow}$
- 4: $pow \leftarrow (pow \cdot x) \bmod p$
- 5: **end for**
- 6: **return** $[E = 1_{\mathbb{G}}]$

Algorithm 4 Verify(pp, st_V, id^*)**Input:** Public parameters $pp = (\mathbb{G}, p, g)$, Verifier state $st_V = (g_0, g_1, \dots, g_n)$, Candidate identifier $id^* \in \mathbb{Z}_p$, policy type Φ_{type} **Output:** Binary decision $b \in \{0, 1\}$ (1 for Accept)

- 1: **if** Φ_{type} is explicit **then**
- 2: **return** VerifyPoly(pp, st_V, id^*)
- 3: **else** ▷ Φ_{type} is Pattern-based
- 4: $\mathcal{P} \leftarrow \text{GetBinPrefixes}(id^*, m)$
- 5: **for each** $prefix \in \mathcal{P}$ **do**
- 6: **if** VerifyPoly($pp, st_V, prefix$) = 1 **then**
- 7: **return** 1
- 8: **end if**
- 9: **end for**
- 10: **return** 0
- 11: **end if**

Hiding Polynomial Degree(Optional). The length of st_V directly reveals the degree n of the committed polynomial $f_\Phi(X)$, which may leak information about the policy (e.g., the size of an explicit whitelist or the complexity of a pattern). To conceal the true degree, the verifier can extend the state vector with dummy group elements: choose $d \geq 0$ and sample $g_{n+1}, \dots, g_{n+d} \xleftarrow{\$} \mathbb{G}$ uniformly at random. These dummy elements are appended to st_V . During verification, the algorithm processes only the first $n + 1$ elements; the dummy elements $g_{n+1}, \dots, g_{n+d}$ are ignored in the actual computation of E . This simple padding technique prevents an adversary from learning the true degree of $f_\Phi(X)$ solely from the size of st_V .

The VerifyPoly algorithm requires $O(n)$ group operations per evaluation, where $n = \deg(f_\Phi)$. For pattern-based policies, at most m calls to VerifyPoly are needed. The security is maintained as the BRC node set $\mathcal{B}$ remains hidden within the polynomial commitment.

4.4 Dynamic Policy Updates

The system supports dynamic policy modifications through the Update protocol defined in Section 3.2. The concrete implementation and the structure of the update instruction Δ differ between explicit whitelists and pattern-based policies.

For **explicit** whitelists S , two update methods are available:

- **Incremental updates:** The Issuer generates an instruction $\Delta = (op, id)$ containing an operation (add or remove) and a single identifier.
- **Complete reissuance:** The Issuer recomputes the polynomial commitment for the new set S' via $\text{Issue}(pp, S')$ and distributes the fresh state st'_V . Here Δ effectively contains st'_V .

For **pattern-based** policies, only complete reissuance is applicable.

This design maintains a consistent Update interface while allowing efficient, policy-specific implementations.

4.4.1 Incremental Updates for Explicit Whitelists. The Update protocol for explicit whitelists is realized through two subroutines: Add (Algorithm 5) for insertion and Remove (Algorithm 6) for deletion. Upon receiving an authorized instruction $\Delta = (op, id)$, the Verifier executes:

$$\text{Update}(pp, st_V, \Delta) = \begin{cases} \text{Add}(pp, st_V, id) & \text{if } op = \text{add} \\ \text{Remove}(pp, st_V, id) & \text{if } op = \text{remove} \end{cases}$$

Both subroutines require only $O(n)$ group operations and are non-interactive, making updates efficient and practical for local execution by the verifier. To minimize information exposure, the Verifier **erases the plaintext identifier** id from memory immediately after completing the update.

The incremental approach minimizes transmission overhead, since only the operation type and a single identifier are sent. However, during local computation, the plaintext id is temporarily exposed in the Verifier's memory, requiring protective measures such as secure enclaves[29–32] or memory encryption[33–35] if update confidentiality is required. In contrast, complete reissuance transmits a full new state st'_V , which is larger but never exposes plaintext policy elements.

Algorithm 5 Add(pp, st_V, id^+)

Input: $pp = (\mathbb{G}, p, g)$, $st_V = (g_0, g_1, \dots, g_n)$, $id^+ \in \mathbb{Z}_p$

Output: Updated state $st'_V = (g'_0, g'_1, \dots, g'_{n+1})$

- 1: Let n be the current degree (st_V has $n + 1$ elements).
 - 2: Compute $st'_V = (g'_0, g'_1, \dots, g'_{n+1})$ as follows:
 - 3: $g'_0 \leftarrow (g_0)^{-id^+}$. $\triangleright$ Compute $g^{-a_0 \cdot id^+} = g^{a'_0}$
 - 4: **for** $k = 1$ **to** n **do**
 - 5: $g'_k \leftarrow g_{k-1} \cdot (g_k)^{-id^+}$.
 - 6: **end for**
 - 7: $g'_{n+1} \leftarrow g_n$.
 - 8: **return** st'_V
-

Algorithm 6 Remove(pp, st_V, id^-)

Input: $pp = (\mathbb{G}, p, g)$, $st_V = (g_0, g_1, \dots, g_n)$, $id^- \in \mathbb{Z}_p$

Output: Updated state $st'_V = (g'_0, g'_1, \dots, g'_{n-1})$

- 1: Let n be the current degree (st_V has $n + 1$ elements).
 - 2: Compute $st'_V = (g'_0, g'_1, \dots, g'_{n-1})$ as follows:
 - 3: $g'_{n-1} \leftarrow g_n$.
 - 4: **for** $k = n - 2$ **down to** 0 **do**
 - 5: $g'_k \leftarrow g_{k+1} \cdot (g'_{k+1})^{id^-}$.
 - 6: **end for**
 - 7: **return** st'_V
-

4.4.2 Complete Reissuance. For pattern-based policies or when incremental updates are not desired for explicit whitelists, the Issuer can perform complete reissuance by executing $\text{Issue}(pp, \Phi)$ (Algorithm 2) to obtain the new state st'_V , and then transmits it to Verifier.

The Verifier replaces its old state with st'_V . This approach ensures consistency and preserves policy confidentiality, as the Verifier never accesses plaintext policy representations.

5 Security Analysis

This section provides formal security proofs for our PEW construction, demonstrating that it satisfies the security definitions given in Section 3.3.

5.1 Proof of Completeness

The completeness property ensures that any identity satisfying the policy will be accepted when verified with the corresponding verifier state. Formally, for all security parameters $\lambda \in \mathbb{N}$, all public parameters $pp \leftarrow \text{Setup}(1^\lambda)$, all policies Φ , and all identities id such that $\Phi(id) = 1$ (i.e., id satisfies Φ), the following holds after generating the verifier state $st_V \leftarrow \text{Issue}(pp, \Phi)$:

$$\Pr[\text{Verify}(pp, st_V, id) = 1] = 1.$$

Completeness follows directly from the construction of the polynomial f_Φ from policy Φ and the correctness of the underlying commitment scheme. Since $f_\Phi(id) = 0$ whenever $\Phi(id) = 1$, the verification equation holds for all valid identities when the cryptographic primitives are correctly instantiated.

5.2 Proof of Soundness

THEOREM 5.1 (SOUNDNESS). *The PEW construction satisfies the soundness property defined in Section 3.3.*

PROOF. Fix the public parameters $pp = (\mathbb{G}, p, g)$ where p is a λ -bit prime. Consider an invalid identifier id^* that does not satisfy the policy Φ .

Case 1: Explicit Whitelist S . Let $f_S(X) = \prod_{id \in S} (X - id)$. Since $id^* \notin S$, the value $N_S \triangleq f_S(id^*) = \prod_{id \in S} (id^* - id)$ is a non-zero integer.

Verification succeeds if and only if $g^{f_S(id^*)} = 1_{\mathbb{G}}$, which requires $N_S \equiv 0 \pmod{p}$. Thus the adversary's success probability in this case is

$$\Pr[p \mid N_S] \approx \frac{1}{p} \approx 2^{-\lambda}.$$

Case 2: Pattern-Based Policy Φ . Let $\mathcal{B}$ be the set of BRCs representing Φ , with $f_\Phi(X) = \prod_{\text{brc} \in \mathcal{B}} (X - \text{brc})$. For invalid id^* , let $\mathcal{P} \leftarrow \text{GetBinPrefixes}(id^*, m)$. Since id^* does not satisfy Φ , $\mathcal{P} \cap \mathcal{B} = \emptyset$, and thus $f_\Phi(\text{prefix}) \neq 0$ for every $\text{prefix} \in \mathcal{P}$.

Verification succeeds iff $\exists \text{prefix} \in \mathcal{P}$ such that $g^{f_\Phi(\text{prefix})} = 1_{\mathbb{G}}$, i.e., $f_\Phi(\text{prefix}) \equiv 0 \pmod{p}$. For each fixed $\text{prefix} \in \mathcal{P}$, $\Pr[p \mid f_\Phi(\text{prefix})] \approx 1/p$. By the union bound over the m prefixes in $\mathcal{P}$:

$$\Pr[\text{success}] \leq \frac{m}{p} \approx m \cdot 2^{-\lambda}.$$

Conclusion. In both cases, the success probability of a cheating prover is negligible in the security parameter λ . Hence the scheme satisfies soundness. $\square$

5.3 Proof of Update Correctness

THEOREM 5.2 (UPDATE CORRECTNESS). *The PEW construction satisfies the update correctness property defined in Section 3.3.*

PROOF. We consider both incremental updates for explicit whitelists and complete reissuance for all policy types.

Case 1: Incremental Updates (Explicit Whitelists). Let $pp = (\mathbb{G}, p, g)$ be public parameters, and let S be an explicit whitelist, $\text{st}_V \leftarrow \text{Issue}(pp, S)$, where $\text{st}_V = (g^{a_0}, \dots, g^{a_n})$ and $f_S(X) = \sum_{k=0}^n a_k X^k$.

Addition: Let $S' = S \cup \{id^+\}$ with $id^+ \notin S$. Then $f_{S'}(X) = f_S(X) \cdot (X - id^+) = \sum_{k=0}^{n+1} a'_k X^k$, where $a'_{n+1} = a_n$. Algorithm 5 computes:

$$g'_0 = (g^{a_0})^{-id^+} = g^{-a_0 id^+} = g^{a'_0},$$

$$g'_k = g^{a_{k-1}} \cdot (g^{a_k})^{-id^+} = g^{a_{k-1} - a_k id^+} = g^{a'_k} \quad (1 \leq k \leq n),$$

$$g'_{n+1} = g^{a_n} = g^{a'_{n+1}}.$$

Thus for any identifier $id \in \mathcal{ID}$, verification under $\text{st}'_V = (g'_0, \dots, g'_{n+1})$ yields the same result as verification under the freshly issued state $\text{st}'_V \leftarrow \text{Issue}(pp, S')$, satisfying the update correctness definition.

Removal: A symmetric argument shows that for any identifier $id \in \mathcal{ID}$, verification under the state st'_V produced by Algorithm 6 yields the same result as verification under a fresh issuance $\text{Issue}(pp, S \setminus \{id^-\})$, thus satisfying update correctness.

Case 2: Complete Reissuance. For any policy Φ and update instruction Δ leading to Φ' , complete reissuance computes $\text{st}'_V \leftarrow \text{Issue}(pp, \Phi')$ directly. By construction, for any identifier $id \in \mathcal{ID}$, verification under st'_V yields the same result as under a fresh issuance on Φ' , thereby satisfying update correctness.

Combined Correctness. Since both update methods produce states that yield identical verification outcomes to fresh issuance, the construction satisfies update correctness for all valid policy modifications. $\square$

5.4 Proof of Policy Confidentiality

THEOREM 5.3 (POLICY CONFIDENTIALITY). *The PEW construction satisfies the policy confidentiality property defined in Section 3.3, under the Discrete Logarithm (DL) assumption in the group $\mathbb{G}$. This holds for both explicit whitelist policies and pattern-based policies.*

PROOF. We consider two cases separately: explicit whitelist policies and pattern-based policies.

Case 1: Explicit Whitelist Policies. Assume a PPT adversary $\mathcal{A}$ wins the policy confidentiality game for an explicit whitelist of size n with advantage $\epsilon(\lambda)$. We construct a simulator $\mathcal{S}$ that uses $\mathcal{A}$ to solve the DL problem.

$\mathcal{S}$ receives a DL challenge (g, g^a) and proceeds as follows:

- (1) **Setup:** $\mathcal{S}$ sets $pp = (\mathbb{G}, p, g)$ and sends it to $\mathcal{A}$.
- (2) **Challenge Construction:** Upon receiving the whitelist size n from $\mathcal{A}$, $\mathcal{S}$ simulates a commitment to a size- n whitelist containing a :
 - Choose $n - 1$ random dummy roots $r_1, \dots, r_{n-1} \xleftarrow{\$} \mathbb{Z}_p^*$.
 - Let $f_\Phi(X) = (X - a) \prod_{i=1}^{n-1} (X - r_i) = X^n + \sum_{k=0}^{n-1} c_k X^k$.
 - Compute the commitment $\text{st}_V = (g^{c_0}, \dots, g^{c_{n-1}}, g)$ by evaluating coefficient expressions in the exponent. For example, for $c_{n-1} = -(a + \sum_{i=1}^{n-1} r_i)$:

$$g^{c_{n-1}} = g^{-(a + \sum_{i=1}^{n-1} r_i)} = (g^a)^{-1} \cdot \prod_{i=1}^{n-1} g^{-r_i}.$$

Other g^{c_k} are computed similarly using $\{r_i\}$ and g^a .

- Send st_V to $\mathcal{A}$.

The distribution of st_V is identical to a real commitment.

- (3) **Query Phase:** $\mathcal{A}$ performs local verification queries using st_V .
- (4) **Output:** $\mathcal{A}$ outputs a fresh identifier id^* .
 - If $g^{id^*} = g^a$, $\mathcal{S}$ outputs id^* .
 - Otherwise, $\mathcal{S}$ outputs $\perp$.

Let $S = \{a, r_1, \dots, r_{n-1}\}$ be the underlying whitelist. Conditioned on $\mathcal{A}$ winning (i.e., $id^* \in S$), we have:

$$\Pr[id^* = a \mid \mathcal{A} \text{ wins}] = \frac{1}{n}.$$

Thus, $\mathcal{S}$'s success probability is $\Pr[\mathcal{S} \text{ solves DL}] = \epsilon(\lambda)/n$. Since n is polynomial in λ , if $\epsilon(\lambda)$ is non-negligible, then $\epsilon(\lambda)/n$ is also non-negligible, contradicting the DL assumption.

Case 2: Pattern-Based Policies. For pattern-based policies, the polynomial $f_\Phi(X)$ is constructed as $f_\Phi(X) = \prod_{\text{brc} \in \mathcal{B}} (X - \text{brc})$, where $\mathcal{B}$ is the BRC node set representing the pattern. Given that the policy accepts exactly n identifiers, the size $|\mathcal{B}|$ of the BRC node set is not fixed but depends on the specific interval structure.

Let D denote the set of all possible BRC sizes for intervals containing exactly n identifiers: $D = \{|\mathcal{B}| : \exists [a, b] \subseteq [0, 2^m - 1] \text{ with } |[a, b]| = n \wedge \mathcal{B} = \text{GetBRC}(a, b, m)\}$.

Note that $D \subseteq \{1, 2, \dots, 2\lceil \log_2 n \rceil\}$ and $|D| = O(\log n)$.

Now assume a PPT adversary $\mathcal{A}_{\text{pat}}$ breaks policy confidentiality for pattern-based policies with advantage $\epsilon_{\text{pat}}(\lambda)$. We construct a simulator $\mathcal{S}$ that uses $\mathcal{A}_{\text{pat}}$ to solve the DL problem:

$\mathcal{S}$ receives a DL challenge (g, g^a) and proceeds similarly to Case 1, with the following modification in the challenge construction step:

Upon receiving n from $\mathcal{A}_{\text{pat}}$, $\mathcal{S}$:

- (a) Computes the set D as defined above.
- (b) Chooses $n_d \xleftarrow{\$} D$ uniformly at random.
- (c) Proceeds as in Case 1, constructing a polynomial of degree n_d with roots $\{a, r_1, \dots, r_{n_d-1}\}$ where $r_1, \dots, r_{n_d-1} \xleftarrow{\$} \mathbb{Z}_p^*$, and computing the corresponding commitment st_V .

The rest of the simulation follows exactly as in Case 1.

Analysis. From $\mathcal{A}_{\text{pat}}$'s perspective, the simulation is perfect: the commitment st_V is identically distributed to a real commitment for a pattern-based policy accepting n identifiers, with the correct distribution of BRC sizes n_d .

Let $S = \{a, r_1, \dots, r_{n_d-1}\}$ be the root set. Conditioned on $\mathcal{A}_{\text{pat}}$ outputting a valid id^* (i.e., id^* is a root of $f_\Phi(X)$), and noting that all roots are exchangeable from the adversary's view, we have:

$$\Pr[id^* = a \mid \mathcal{A}_{\text{pat}} \text{ succeeds}] = \frac{1}{n_d}.$$

Thus, $\mathcal{S}$'s success probability is at least:

$$\Pr[\mathcal{S} \text{ solves DL}] \geq \frac{\epsilon_{\text{pat}}(\lambda)}{\max(D)}.$$

Since $\max(D) = O(\log n)$ and $n = \text{poly}(\lambda)$, if $\epsilon_{\text{pat}}(\lambda)$ is non-negligible, then $\epsilon_{\text{pat}}(\lambda)/O(\log n)$ is also non-negligible, contradicting the DL assumption.

Thus, both explicit whitelist and pattern-based policies are secure under the DL assumption, completing the proof. $\square$

6 Performance Evaluation

This section presents a comprehensive evaluation of our Privacy-Enhanced Whitelist (PEW) scheme. We first conduct a theoretical analysis comparing the functional characteristics of our construction against two baseline approaches: encrypted whitelists, and hashed whitelists. Subsequently, we provide experimental measurements from a prototype implementation to assess practical performance.

6.1 Theoretical Analysis

Table 1: Comparison of whitelist schemes

Property	Encrypted Whitelist	Hash-based Whitelist	Our Scheme
Storage Confidentiality	Yes	Yes	Yes
Computation Confidentiality	No	Yes	Yes
Expressive Policies	Yes	No	Yes

A comparison of privacy-enhancing approaches for whitelists is summarized in Table 1. The analysis reveals a fundamental trade-off between confidentiality and expressiveness in existing methods, which our scheme resolves.

Encrypted whitelists provide storage confidentiality but fail during computation: verification requires decryption, exposing plaintext identifiers in memory. The decryption key becomes a single point of failure—its compromise reveals the entire policy. While pattern-based matching is possible after decryption, this sacrifices computational privacy.

Hash-based whitelists achieve both storage and computation confidentiality by operating only on hashed values. This eliminates key exposure risks but fundamentally limits expressiveness to exact matching: cryptographic hashing destroys structural relationships, making pattern-based policies infeasible.

Our Privacy-Enhanced Whitelist construction overcomes these limitations. It provides both storage and computation confidentiality through polynomial commitments, enabling verification without plaintext exposure. Crucially, it also supports expressive pattern-based policies via integration with the Best Range-Covering technique, encoding ranges, prefixes, and suffixes within the same cryptographic framework. Security is formally proven under the Discrete Logarithm assumption, ensuring policy confidentiality even against a compromised verifier. This unified approach makes our scheme suitable for sensitive applications requiring both strong privacy and flexible authorization policies.

6.2 Experimental Evaluation

We implement a single-threaded prototype of our Privacy-Enhanced Whitelists system in C (C11 standard) using the OpenSSL library for secp256k1 elliptic curve operations. The implementation is compiled with GCC 11.4 with optimization flags `-O3 -march=native`. Experiments were conducted on an Intel Xeon Platinum 8358P system (2.60GHz, 32 cores) with 1TB RAM running Ubuntu Linux 22.04. The single-threaded design isolates the fundamental algorithmic costs of core cryptographic operations, enables precise performance measurement. While this baseline implementation already demonstrates practical efficiency, significant performance improvements remain achievable through parallelism, algorithmic optimizations, and hardware acceleration.

Storage Overhead Analysis. For explicit whitelists, the primary storage cost is the verifier state $\text{st}_V = (g^{a_0}, g^{a_1}, \dots, g^{a_n})$, which encodes the polynomial $f_S(X) = \prod_{id \in S} (X - id)$ in the exponent. Each element g^{a_k} is a point on the secp256k1 curve, represented in compressed format requiring 33 bytes. Therefore, for an explicit whitelist of size n , the state size is:

$$|\text{st}_V| = 33(n + 1) \text{ bytes.}$$

The linear scaling $O(n)$ ensures that storage remains manageable even for large whitelists—for example, 33 KB for 1,000 identifiers or 330 KB for 10,000 identifiers.

For pattern-based policies, storage is significantly more efficient due to the compact Best Range-Covering (BRC) representation. A pattern policy Φ (range, prefix, or suffix) is encoded as $f_\Phi(X) = \prod_{\text{brc} \in \mathcal{B}} (X - \text{brc})$, where $\mathcal{B}$ is the BRC node set. The size $|\mathcal{B}|$ is logarithmic in the number of identifiers accepted by the pattern: for an interval of size n , $|\mathcal{B}| = O(\log n)$. Consequently, the verifier state size becomes:

$$|\text{st}_V| = 33(|\mathcal{B}| + 1) \approx O(\log n) \text{ bytes.}$$

For typical patterns, this translates to tens to hundreds of bytes regardless of how many identifiers the pattern accepts. For instance, a /24 IPv4 prefix rule (accepting 256 addresses) requires only one BRC node ($|\mathcal{B}| = 1$), resulting in storage of 66 bytes.

Thus, our system accommodates both storage patterns: linear growth for explicit whitelists and logarithmic efficiency for pattern-based policies, making it adaptable to diverse deployment needs—from small arbitrary whitelists to large, pattern-based authorization sets.

6.2.1 Explicit Whitelist Performance. We evaluated explicit whitelists with sizes from 200 to 2000 identifiers, measuring the computational

costs of issuance, membership verification, and dynamic updates. Results are shown in Figures 3, 4, and 5.

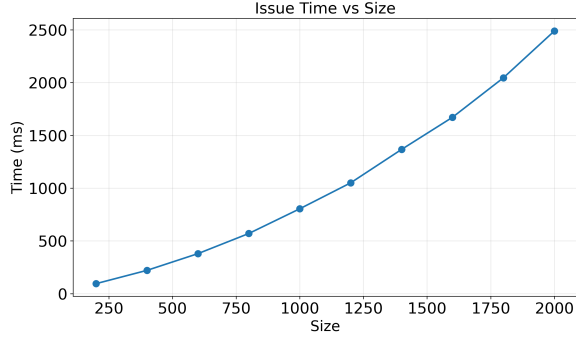


Figure 3: Issue time versus whitelist size

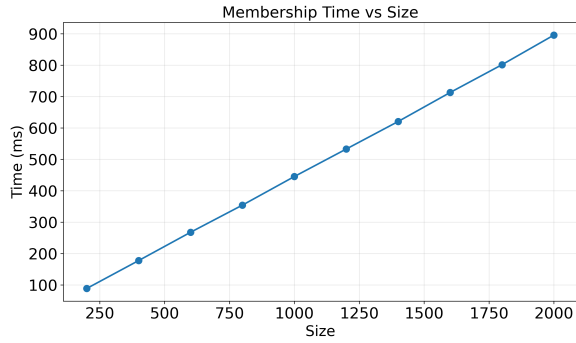


Figure 4: Membership verification time versus whitelist size

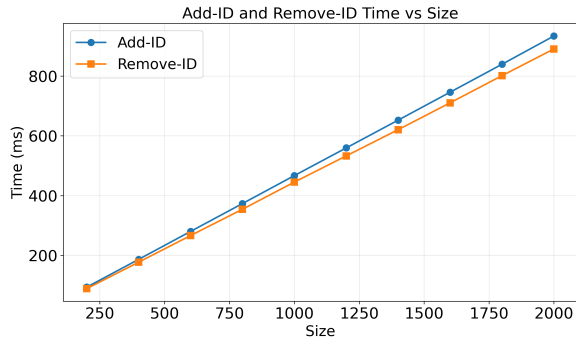


Figure 5: Update operation times versus whitelist size

Figure 3 shows the Issue time as a function of whitelist size. The curve exhibits clear quadratic growth, increasing from 95 ms for $n = 200$ to 2480 ms for $n = 2000$. This matches the expected $O(n^2)$ complexity of constructing the polynomial $f_S(X) = \prod_{id \in S} (X - id)$ and computing its coefficient commitments. Although issuance exhibits quadratic complexity, it is typically performed only when policies are created or significantly modified—not during routine

verification. Therefore, even for whitelists with several thousand identifiers, the issuance overhead remains acceptable in practice.

Membership verification time, shown in Figure 4, demonstrates linear scaling with whitelist size. Execution time increases from 90 ms at $n = 200$ to 891 ms at $n = 2000$, closely following the theoretical $O(n)$ complexity of homomorphic polynomial evaluation. Each verification requires n group exponentiations to compute $g^{f(id^*)}$ from the commitments. With sub-second verification for whitelists of up to 2000 identifiers, the scheme proves practical for moderately sized deployments.

Figure 5 compares the costs of adding and removing identifiers. Both operations scale linearly with n , consistent with their $O(n)$ group operation requirements. Addition (95–935 ms) is marginally more expensive than removal (89–891 ms) due to the extra polynomial multiplication step in the Add algorithm. The nearly parallel trend lines indicate that both operations maintain similar computational characteristics, with the performance gap remaining stable at approximately 5% across all tested sizes. This efficiency enables dynamic updates for explicit whitelists without requiring complete reissuance, offering a lightweight alternative for scenarios where incremental policy changes are frequent.

6.2.2 Pattern-Based Whitelist Performance.

Note on Match vs Mismatch Measurement. Unlike explicit whitelists where verification cost is uniform for all identifiers, pattern-based verification exhibits asymmetric performance: matches may complete early once a matching prefix is found, while mismatches require evaluating all m prefixes. This distinction justifies our separate reporting of match and mismatch times.

Note on Implementation . To ensure correct polynomial evaluation for pattern-based policies, BRC nodes and candidate prefixes are encoded using a fixed mapping (0→01, 1→10) to avoid ambiguity (e.g., between "01" and "1"). This ensures correct matching at the cost of a constant-factor increase in string length, which is reflected in the reported evaluation times.

For pattern-based whitelists, performance depends on three key parameters: m (identifier bit-length), n (size of the BRC node set $\mathcal{B}$), and pos (the prefix position where a match occurs). Experimental results reveal clear linear relationships in issuance and verification costs.

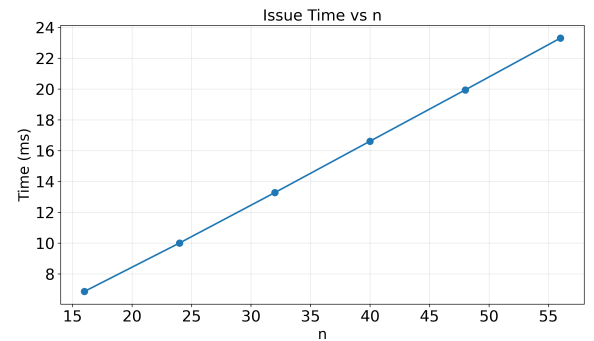


Figure 6: Issue time versus BRC node set size n

Issue time exhibits linear scaling with n , as shown in Figure 6. For fixed $m = 64$, Issue cost increases consistently from approximately 7 ms at $n = 16$ to 23 ms at $n = 56$, demonstrating the lightweight nature of BRC-based policy encoding.

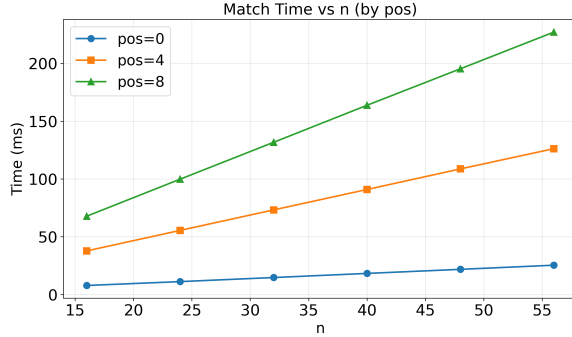


Figure 7: Membership verification time for successful matches

Match verification performance varies significantly with both n and pos , as illustrated in Figure 7. When a match occurs at the first prefix position ($pos = 0$), verification completes rapidly, increasing only from 8 ms ($n = 16$) to 25 ms ($n = 56$). However, deeper matches ($pos = 8$) incur substantially higher costs, rising from 68 ms to 228 ms over the same n range. This confirms that early prefix matches are crucial for minimizing verification latency.

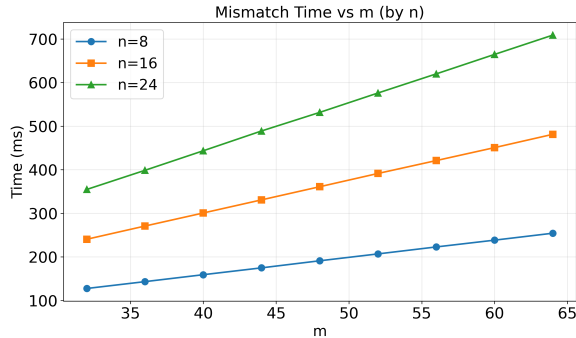


Figure 8: Verification time for mismatches

Mismatch verification, shown in Figure 8, displays linear dependence on both m and n . For a fixed $n = 8$, mismatch time doubles from 128 ms ($m = 32$) to 255 ms ($m = 64$), reflecting the need to check all m prefixes. Similarly, at fixed $m = 64$, time grows from 255 ms ($n = 8$) to 708 ms ($n = 24$) due to the increased polynomial degree. This predictable scaling enables accurate performance estimation in deployment scenarios.

Practical Implications. Our evaluation demonstrates the practical viability of both explicit and pattern-based whitelist schemes. For explicit whitelists, the system delivers usable performance for sizes up to several thousand identifiers. The quadratic issuance cost is incurred only when a policy is first created or substantially modified, while verification and updates maintain linear complexity,

supporting efficient runtime operation. For pattern-based policies, verification completes within tens of milliseconds for early matches, while mismatches incur costs proportional to identifier length m , with the compact BRC representation ensuring logarithmic storage overhead. These results confirm that strong cryptographic privacy—protecting both storage and computation—can be achieved without prohibitive performance overheads in realistic deployment scenarios.

It is important to note that the current implementation is single-threaded and uses no specialized optimizations. Since core operations such as elliptic curve exponentiations are highly parallelizable, significant performance improvements remain achievable through multi-threading, batched processing, and hardware acceleration. Therefore, the proposed system not only meets practical performance requirements in its current form, but also holds substantial potential for further optimization in production environments.

7 Conclusion

In this paper, we have introduced and formalized the concept of Privacy-Enhanced Whitelists (PEW), a novel access control framework that decouples policy enforcement from policy exposure. By shifting trust from the verifier’s infrastructure to cryptographic verification, our scheme eliminates the traditional single point of failure—plaintext policy storage at the verifier—while maintaining support for both explicit enumeration and expressive pattern-based policies.

We presented a formal security model that rigorously defines policy confidentiality against a compromised verifier, a property not achieved by prior encryption- or hash-based approaches. Our concrete construction, based on polynomial commitments in elliptic curve groups, provides an efficient and unified mechanism for encoding and verifying policies without exposing their content. The scheme naturally supports dynamic updates and is proven secure under the standard Discrete Logarithm assumption.

Through comprehensive evaluation, we demonstrated the practical feasibility of our approach. For explicit whitelists, verification and update operations scale linearly with list size. Pattern-based policies benefit from the compact Best Range-Covering representation, achieving efficient storage and verification. Even with a single-threaded, unoptimized implementation, the system achieves sub-second performance for realistic policy sizes, confirming its deployability in real-world scenarios.

Looking forward, several directions remain open for future work. These include extending the framework to support more complex policy languages, exploring post-quantum secure instantiations, and investigating decentralized or distributed issuer models. The efficient base construction also invites further optimization through parallelism, hardware acceleration, and integration with trusted execution environments.

In summary, this work establishes a foundational and practical framework for privacy-preserving access control, enabling high-assurance whitelist enforcement without sacrificing policy confidentiality—a critical step forward for securing sensitive infrastructures where the authorization policy itself must remain a protected secret.

References

- [1] Samarati P, De Vimercati S C. Access control: Policies, models, and mechanisms[M]//International school on foundations of security analysis and design. Berlin, Heidelberg: Springer Berlin Heidelberg, 2000: 137-196.
- [2] Bertino E, Ghinita G, Kamra A. Access control for databases: Concepts and systems[J]. Foundations and Trends® in Databases, 2011, 3(1-2): 1-148.
- [3] Tourani R, Misra S, Mick T, et al. Security, privacy, and access control in information-centric networking: A survey[J]. IEEE communications surveys & tutorials, 2017, 20(1): 566-600.
- [4] Pareek H, Romana S, Eswari P R L. Application whitelisting: approaches and challenges[J]. International Journal of Computer Science, Engineering and Information Technology (IJCSSEIT), 2012, 2(5): 13-18.
- [5] Leander A. The politics of whitelisting: Regulatory work and topologies in commercial security[J]. Environment and Planning D: Society and Space, 2016, 34(1): 48-66.
- [6] Azeez N A, Misra S, Margaret I A, et al. Adopting automated whitelist approach for detecting phishing attacks[J]. Computers & Security, 2021, 108: 102328.
- [7] Morales D, Agudo I, Lopez J. Private set intersection: A systematic literature review[J]. Computer Science Review, 2023, 49: 100567.
- [8] Pinkas B, Schneider T, Zohner M. Scalable private set intersection based on OT extension[J]. ACM Transactions on Privacy and Security (TOPS), 2018, 21(2): 1-35.
- [9] Chen H, Laine K, Rindal P. Fast private set intersection from homomorphic encryption[C]//Proceedings of the 2017 ACM SIGSAC conference on computer and communications security. 2017: 1243-1255.
- [10] Dong C, Chen L, Wen Z. When private set intersection meets big data: an efficient and scalable protocol[C]//Proceedings of the 2013 ACM SIGSAC conference on Computer & communications security. 2013: 789-800.
- [11] De Cristofaro E, Tsudik G. Practical private set intersection protocols with linear complexity[C]//International Conference on Financial Cryptography and Data Security. Berlin, Heidelberg: Springer Berlin Heidelberg, 2010: 143-159.
- [12] Bay A, Erkin Z, Hoepman J H, et al. Practical multi-party private set intersection protocols[J]. IEEE Transactions on Information Forensics and Security, 2021, 17: 1-15.
- [13] Huang Y, Evans D, Katz J. Private set intersection: Are garbled circuits better than custom protocols?[C]//NDSS. 2012.
- [14] Ghosh S, Nilges T. An algebraic approach to maliciously secure private set intersection[C]//Annual international conference on the theory and applications of cryptographic techniques. Cham: Springer International Publishing, 2019: 154-185.
- [15] Bianchi G, Rajabi H, Caponi A, et al. Conditional disclosure of encrypted whitelists for DDoS attack mitigation[C]//2013 IEEE Globecom Workshops (GC Wkshps). IEEE, 2013: 200-206.
- [16] Pryde J, Angeles N, Carinan S K. Dynamic Whitelisting Using Locality Sensitive Hashing[C]//Pacific-Asia Conference on Knowledge Discovery and Data Mining. Cham: Springer International Publishing, 2018: 181-185.
- [17] McCurley K S. The discrete logarithm problem[C]//Proc. of Symp. in Applied Math. 1990, 42: 49-74.
- [18] Corrigan-Gibbs H, Kogan D. The discrete-logarithm problem with preprocessing[C]//Annual International Conference on the Theory and Applications of Cryptographic Techniques. Cham: Springer International Publishing, 2018: 415-447.
- [19] Diem C. On the discrete logarithm problem in elliptic curves[J]. Compositio Mathematica, 2011, 147(1): 75-104.
- [20] Odlyzko A. Discrete logarithms: The past and the future[J]. Designs, Codes and Cryptography, 2000, 19(2): 129-145.
- [21] Galbraith S D, Gaudry P. Recent progress on the elliptic curve discrete logarithm problem[J]. Designs, Codes and Cryptography, 2016, 78(1): 51-72.
- [22] Kate A, Zaverucha G M, Goldberg I. Constant-size commitments to polynomials and their applications[C]//International conference on the theory and application of cryptography and information security. Berlin, Heidelberg: Springer Berlin Heidelberg, 2010: 177-194.
- [23] Boneh D, Drake J, Fisch B, et al. Efficient polynomial commitment schemes for multiple points and polynomials[J]. Cryptology ePrint Archive, 2020.
- [24] Zhang J, Xie T, Hoang T, et al. Polynomial commitment with a One-to-Many prover and applications[C]//31st USENIX Security Symposium (USENIX Security 22). 2022: 2965-2982.
- [25] Demertzis, S.Papaopoulos, O.Papapetrou, A.Deligianakis, and M.N. Garofalakis. Practical private range search revisited. In SIGMOD, 2016.
- [26] Zuo, C., et al. Dynamic Searchable Symmetric Encryption Schemes Supporting Range Queries with Forward/Backward Privacy. In ESORICS 2018.
- [27] WangJiafan, ChowSherman. Forward and Backward-Secure Range-Searchable Symmetric Encryption. Proceedings on Privacy Enhancing Technologies, 2022(1):28-48.
- [28] Kiayias A, Papadopoulos S, Triandopoulos N, et al. Delegatable pseudorandom functions and applications. ACM CCS2013, pp.669-684
- [29] Zhao S, Zhang Q, Qin Y, et al. Sectee: A software-based approach to secure enclave architecture using tee[C]//Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security. 2019: 1723-1740.
- [30] Ferraiuolo A, Baumann A, Hawblitzel C, et al. Komodo: Using verification to disentangle secure-enclave hardware from software[C]//Proceedings of the 26th Symposium on Operating Systems Principles. 2017: 287-305.
- [31] Bourgeat T, Lebedev I, Wright A, et al. Mi6: Secure enclaves in a speculative out-of-order processor[C]//Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture. 2019: 42-56.
- [32] Weisse O, Bertacco V, Austin T. Regaining lost cycles with HotCalls: A fast interface for SGX secure enclaves[J]. ACM SIGARCH Computer Architecture News, 2017, 45(2): 81-93.
- [33] Kaplan D, Powell J, Woller T. AMD memory encryption[J]. White paper, 2016, 13: 12.
- [34] Henson M, Taylor S. Memory encryption: A survey of existing techniques[J]. ACM Computing Surveys (CSUR), 2014, 46(4): 1-26.
- [35] Gueron S. Memory encryption for general-purpose processors[J]. IEEE Security & Privacy, 2016, 14(6): 54-62.